LAB ASSIGNMENT-12.3

SINDHU KATAKAM

2303A52481

BATCH-41

Task 1: Sorting Student Records for Placement Drive Scenario

Prompt:

Write a Python program to sort student records for a placement drive. Each student record should have the student's name, roll number, and CGPA. Use Quick Sort and Merge Sort algorithms to arrange the records based on CGPA. Create a function to store all student records in a list and show how the sorting works using real-life sample data. The code should be neat, simple, and well-commented so it is easy to understand. Also, check and compare the running time of both sorting methods using a large dataset. Finally, add a function to display the top 10 students with the highest CGPA.

Code:

```python
# Student Placement Record Sorting using Quick Sort and Merge Sort
# Each student has: name, roll number, CGPA
# The program sorts by CGPA, compares runtimes, and displays top 10 students

import random
import string
import time

# Define the Student record as a dictionary for simplicity
def create_student(name, roll, cgpa):
    return {'name': name, 'roll': roll, 'cgpa': cgpa}

# Function to generate random student data
def generate_students(num):
    students = []
    for i in range(num):
        name = ''.join(random.choices(string.ascii_uppercase, k=5))
        roll = f"R{i+1:05d}"
        cgpa = round(random.uniform(5.0, 10.0), 2)
        students.append(create_student(name, roll, cgpa))
    return students

# Quick Sort implementation for student records by CGPA
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]['cgpa']
    left = [x for x in arr if x['cgpa'] > pivot]  # Descending order
    middle = [x for x in arr if x['cgpa'] == pivot]
    right = [x for x in arr if x['cgpa'] < pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

```python
# Merge Sort implementation for student records by CGPA
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i]['cgpa'] > right[j]['cgpa']:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Function to display student records
def display_students(students, title="Student Records"):
    print(f"\n{title}:")
    print(f"{'Name':<10} {'Roll':<8} {'CGPA':<5}")
    print('-'*26)
    for s in students:
        print(f"{s['name']:<10} {s['roll']:<8} {s['cgpa']:<5}")
```

```python
# Function to display top 10 students
def display_top_10(students):
    top_10 = students[:10]
    display_students(top_10, title="Top 10 Students by CGPA")

def main():
    # Sample data for demonstration
    sample_students = [
        create_student('Alice', 'R00001', 9.2),
        create_student('Bob', 'R00002', 8.7),
        create_student('Charlie', 'R00003', 9.5),
        create_student('David', 'R00004', 7.8),
        create_student('Eva', 'R00005', 8.9),
        create_student('Frank', 'R00006', 9.1),
        create_student('Grace', 'R00007', 8.5),
        create_student('Helen', 'R00008', 9.7),
        create_student('Ivy', 'R00009', 7.9),
        create_student('Jack', 'R00010', 8.3),
    ]
    print("--- Sample Data Sorting Demo ---")
    display_students(sample_students, "Original Records")

    # Quick Sort
    quick_sorted = quick_sort(sample_students)
    display_students(quick_sorted, "Quick Sort Result")

    # Merge Sort
    merge_sorted = merge_sort(sample_students)
    display_students(merge_sorted, "Merge Sort Result")

    display_top_10(quick_sorted)
```

```python
    # Performance comparison with large dataset
    print("\n--- Performance Comparison on Large Dataset ---")
    large_students = generate_students(10000)
    t1 = time.time()
    quick_sort(large_students)
    t2 = time.time()
    print(f"Quick Sort Time: {t2-t1:.4f} seconds")

    t3 = time.time()
    merge_sort(large_students)
    t4 = time.time()
    print(f"Merge Sort Time: {t4-t3:.4f} seconds")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS D:\AI_ASSIT_CODING> & C:\Users
0.3.py
--- Sample Data Sorting Demo ---

Original Records:
Name       Roll     CGPA
--------------------------
Alice      R00001   9.2
Bob        R00002   8.7
Charlie    R00003   9.5
David      R00004   7.8
Eva        R00005   8.9
Frank      R00006   9.1
Grace      R00007   8.5
Helen      R00008   9.7
Ivy        R00009   7.9
Jack       R00010   8.3

Quick Sort Result:
Name       Roll     CGPA
--------------------------
Helen      R00008   9.7
Charlie    R00003   9.5
Alice      R00001   9.2
Frank      R00006   9.1
Eva        R00005   8.9
Bob        R00002   8.7
Grace      R00007   8.5
Jack       R00010   8.3
Ivy        R00009   7.9
David      R00004   7.8

Merge Sort Result:
Name       Roll     CGPA
--------------------------
Helen      R00008   9.7
Charlie    R00003   9.5
Alice      R00001   9.2
Frank      R00006   9.1
Eva        R00005   8.9
Bob        R00002   8.7
Grace      R00007   8.5
Jack       R00010   8.3
Ivy        R00009   7.9
David      R00004   7.8

Top 10 Students by CGPA:
Name       Roll     CGPA
--------------------------
Helen      R00008   9.7
Charlie    R00003   9.5
Alice      R00001   9.2
Frank      R00006   9.1
Eva        R00005   8.9
Bob        R00002   8.7
Grace      R00007   8.5
Jack       R00010   8.3
Ivy        R00009   7.9
David      R00004   7.8

--- Performance Comparison on Large Dataset ---
Quick Sort Time: 0.0415 seconds
Merge Sort Time: 0.0685 seconds
```

EXPLANATION:

I wrote this code to help organize student records efficiently for a placement drive. By using both Quick Sort and Merge Sort, I can compare which sorting method works faster with large amounts of data. The code is simple and well-commented so anyone can understand how it works. I also included a function to display the top 10 students with the highest CGPA, making it easy to identify the best performers. This program is useful for quickly sorting and viewing student data in a real-world scenario.

Task 2: Implementing Bubble Sort with AI Comments

Prompt:

Write a Python program to implement the Bubble Sort algorithm with proper comments. Add inline comments in the code to clearly explain important steps like comparing elements, swapping them, completing each pass, and stopping the loop when the list is already sorted. The code should be simple, clean, and easy to understand. Also, explain the time complexity of the Bubble Sort algorithm in simple words.

CODE:

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        swapped = False  # Flag to check if any swapping happened in this pass
        # Last i elements are already in place, so we don't need to check them
        for j in range(0, n - i - 1):
            # Compare the current element with the next element
            if arr[j] > arr[j + 1]:
                # Swap if the element found is greater than the next element
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True  # Set flag to True if swapping happened
        # One pass completed
        # If no two elements were swapped in the inner loop, then the list is already sorted
        if not swapped:
            break  # Stop the loop early if the list is sorted

# Sample usage
if __name__ == "__main__":
    data = [64, 34, 25, 12, 22, 11, 90]
    print("Original list:", data)
    bubble_sort(data)
    print("Sorted list:", data)

# Time Complexity Explanation:
# Bubble Sort compares each pair of adjacent elements and swaps them if they are in the wrong order.
# In the worst and average case, it needs to go through the list n times, and each time it compares up to r
# So, the time complexity is O(n^2), which means it gets much slower as the list gets bigger.
```

OUTPUT:

```
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
```

EXPLANATION:

I wrote this code to help sort a list of numbers in a simple way using the Bubble Sort algorithm. I added clear comments so anyone can easily follow each step, like comparing and swapping elements. The code is easy to read and understand, which is helpful for beginners learning sorting algorithms. I also explained the time complexity in simple words to show why Bubble Sort is not very fast for large lists. This program is a good starting point for understanding how sorting works in Python.

Task 3: Quick Sort and Merge Sort Comparison

PROMPT:

Write a Python program to implement Quick Sort and Merge Sort using recursion. Add sample function calls to show that both sorting methods work correctly. The code should be clean, simple, and well-commented so that the logic is easy to understand. Also, explain the best case, average case, and worst case time complexities of both algorithms in simple words.

CODE:

```python
# Quick Sort implementation using recursion
def quick_sort(arr):
    # Base case: if the list is empty or has one element, it's already sorted
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[0]  # Choose the first element as the pivot
        left = [x for x in arr[1:] if x < pivot]   # Elements less than pivot
        right = [x for x in arr[1:] if x >= pivot] # Elements greater or equal to pivot
        # Recursively sort left and right, then combine with pivot
        return quick_sort(left) + [pivot] + quick_sort(right)

# Merge Sort implementation using recursion
def merge_sort(arr):
    # Base case: if the list is empty or has one element, it's already sorted
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2  # Find the middle point
    left = merge_sort(arr[:mid])   # Recursively sort the left half
    right = merge_sort(arr[mid:])  # Recursively sort the right half
    return merge(left, right)      # Merge the sorted halves

def merge(left, right):
    result = []
    i = j = 0
    # Merge the two sorted lists into result
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
```

```
            j += 1
    # Add any remaining elements from left or right
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Sample function calls to test both sorting methods
if __name__ == "__main__":
    data1 = [38, 27, 43, 3, 9, 82, 10]
    data2 = data1.copy()

    print("Original list:", data1)
    print("Quick Sort result:", quick_sort(data1))
    print("Merge Sort result:", merge_sort(data2))
```

OUTPUT:

```
Original list: [38, 27, 43, 3, 9, 82, 10]
Quick Sort result: [3, 9, 10, 27, 38, 43, 82]
Merge Sort result: [3, 9, 10, 27, 38, 43, 82]
```

JUSTIFICATION:

I wrote this code to show how Quick Sort and Merge Sort work using recursion to sort a list of numbers. I included clear comments to make each step easy to understand for anyone learning these algorithms. The sample function calls help prove that both sorting methods give correct results. I also explained the time complexities in simple words so it's easy to compare their performance. This program is useful for understanding and practicing two important sorting techniques in Python

Task 4 (Real-Time Application – Inventory Management System)

PROMPT:

Write a Python program for a real-time Inventory Management System. The system should manage many products, and each product should have details like Product ID, name, price, and stock quantity. Add features to quickly search for a product using its ID or name, and also sort the products by price or quantity for stock analysis. Suggest the most suitable searching and sorting algorithms for this system and explain why they are the best choice based on the size of the data, how often it is updated, and performance needs. Show the final answer in a table format that includes the operation, recommended algorithm, and justification. The code should be clean, simple, and well-commented so it is easy to understand.

CODE:

```python
class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"ID: {self.product_id}, Name: {self.name}, Price: {self.price}, Quantity: {self.quantity}"

class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    # Search product by ID using Linear Search
    def search_by_id(self, product_id):
        for product in self.products:
            if product.product_id == product_id:
                return product
        return None

    # Search product by Name using Linear Search
    def search_by_name(self, name):
        for product in self.products:
            if product.name.lower() == name.lower():
                return product
        return None
```

```python
    # Sort products by price using Python's built-in TimSort (efficient for real-world data)
    def sort_by_price(self):
        self.products.sort(key=lambda x: x.price)

    # Sort products by quantity using Python's built-in TimSort
    def sort_by_quantity(self):
        self.products.sort(key=lambda x: x.quantity)

    # Display all products
    def display_products(self):
        for product in self.products:
            print(product)

# Sample usage
if __name__ == "__main__":
    inventory = Inventory()
    # Adding sample products
    inventory.add_product(Product(101, "Laptop", 50000, 10))
    inventory.add_product(Product(102, "Mouse", 500, 100))
    inventory.add_product(Product(103, "Keyboard", 1500, 50))
    inventory.add_product(Product(104, "Monitor", 8000, 20))

    print("All Products:")
    inventory.display_products()

    print("\nSearch by ID (102):")
    print(inventory.search_by_id(102))

    print("\nSearch by Name ('Monitor'):")
    print(inventory.search_by_name("Monitor"))
```

```python
    print("\nProducts sorted by Price:")
    inventory.sort_by_price()
    inventory.display_products()

    print("\nProducts sorted by Quantity:")
    inventory.sort_by_quantity()
    inventory.display_products()

# --- Algorithm Recommendation Table ---

from tabulate import tabulate

table = [
    ["Search by ID/Name", "Linear Search", "Simple, works well for small/medium lists, no need to keep data sorted, easy to update."],
    ["Sort by Price/Quantity", "TimSort (Python's sort)", "Very fast for real-world data, stable, handles frequent updates efficiently."]
]

print("\nAlgorithm Recommendation Table:")
print(tabulate(table, headers=["Operation", "Recommended Algorithm", "Justification"]))
```

OUTPUT:

```
All Products:
ID: 101, Name: Laptop, Price: 50000, Quantity: 10
ID: 102, Name: Mouse, Price: 500, Quantity: 100
ID: 103, Name: Keyboard, Price: 1500, Quantity: 50
ID: 104, Name: Monitor, Price: 8000, Quantity: 20

Search by ID (102):
ID: 102, Name: Mouse, Price: 500, Quantity: 100

Search by Name ('Monitor'):
ID: 104, Name: Monitor, Price: 8000, Quantity: 20

Products sorted by Price:
ID: 102, Name: Mouse, Price: 500, Quantity: 100
ID: 103, Name: Keyboard, Price: 1500, Quantity: 50
ID: 104, Name: Monitor, Price: 8000, Quantity: 20
ID: 101, Name: Laptop, Price: 50000, Quantity: 10

Products sorted by Quantity:
ID: 101, Name: Laptop, Price: 50000, Quantity: 10
ID: 104, Name: Monitor, Price: 8000, Quantity: 20
ID: 103, Name: Keyboard, Price: 1500, Quantity: 50
ID: 102, Name: Mouse, Price: 500, Quantity: 100


Algorithm Recommendation Table:
Operation                Recommended Algorithm    Justification
-----------------------  ----------------------   ------------------------------------------------------------------
Search by ID/Name        Linear Search            Simple, works well for small/medium lists, no need to keep data sort
Sort by Price/Quantity   TimSort (Python's sort)  Very fast for real-world data, stable, handles frequent updates effi
```

JUSTIFICATION:

I created this code to help manage products in an inventory system easily. It lets users add products, search for them by ID or name, and sort them by price or quantity for better stock analysis. I used simple searching and sorting methods that work well for small to medium

lists and are easy to update. The code is clean and has comments so anyone can understand how it works. This program is useful for keeping track of products and making inventory management simple.

Task 5: Real-Time Stock Data Sorting & Searching

PROMPT:

Write a Python program for real-time stock data sorting and searching. The program should quickly sort stocks based on their daily gain or loss and allow efficient searching of specific stock symbols. Use Heap Sort to rank stocks by their percentage change and use Hash Maps to instantly retrieve stock data when a stock symbol is entered. The code should be clean, simple, efficient, and easy to understand.

CODE:

```python
# Stock class to hold stock data
class Stock:
    def __init__(self, symbol, name, price, percent_change):
        self.symbol = symbol
        self.name = name
        self.price = price
        self.percent_change = percent_change  # Daily gain/loss in %

    def __repr__(self):
        return f"{self.symbol} ({self.name}): Price={self.price}, Change={self.percent_change}%"

# Heap Sort to sort stocks by percentage change (descending)
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    # Compare left child
    if left < n and arr[left].percent_change > arr[largest].percent_change:
        largest = left
    # Compare right child
    if right < n and arr[right].percent_change > arr[largest].percent_change:
        largest = right
    # Swap and continue heapifying if root is not largest
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(stocks):
    n = len(stocks)
    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
```

```python
        heapify(stocks, n, i)
    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        stocks[0], stocks[i] = stocks[i], stocks[0]
        heapify(stocks, i, 0)
    # Reverse for descending order
    stocks.reverse()

# Hash Map for instant stock lookup
class StockManager:
    def __init__(self):
        self.stocks = []
        self.stock_map = {}  # symbol -> Stock

    def add_stock(self, stock):
        self.stocks.append(stock)
        self.stock_map[stock.symbol] = stock

    def search_by_symbol(self, symbol):
        return self.stock_map.get(symbol, None)

    def sort_by_percent_change(self):
        heap_sort(self.stocks)

    def display_stocks(self):
        for stock in self.stocks:
            print(stock)

if __name__ == "__main__":
    manager = StockManager()
    # Add sample stocks
    manager.add_stock(Stock("AAPL", "Apple Inc.", 175.64, 2.5))
    manager.add_stock(Stock("GOOGL", "Alphabet Inc.", 2820.45, -1.2))
    manager.add_stock(Stock("TSLA", "Tesla Inc.", 720.30, 4.1))
    manager.add_stock(Stock("AMZN", "Amazon.com Inc.", 3342.88, 0.8))

    print("All Stocks:")
    manager.display_stocks()

    print("\nStocks sorted by daily % change (descending):")
    manager.sort_by_percent_change()
    manager.display_stocks()

    print("\nSearch for stock symbol 'TSLA':")
    print(manager.search_by_symbol("TSLA"))
```

OUTPUT:

```
All Stocks:
AAPL (Apple Inc.): Price=175.64, Change=2.5%
GOOGL (Alphabet Inc.): Price=2820.45, Change=-1.2%
TSLA (Tesla Inc.): Price=720.3, Change=4.1%
AMZN (Amazon.com Inc.): Price=3342.88, Change=0.8%

Stocks sorted by daily % change (descending):
TSLA (Tesla Inc.): Price=720.3, Change=4.1%
AAPL (Apple Inc.): Price=175.64, Change=2.5%
AMZN (Amazon.com Inc.): Price=3342.88, Change=0.8%
GOOGL (Alphabet Inc.): Price=2820.45, Change=-1.2%

Search for stock symbol 'TSLA':
TSLA (Tesla Inc.): Price=720.3, Change=4.1%
```

JUSTIFICATION:

I wrote this code to help manage and analyze real-time stock data easily. It uses Heap Sort to quickly rank stocks by their daily percentage change, making it simple to see which stocks gained or lost the most. I used a hash map so anyone can instantly search for a stock by its symbol without waiting. The code is clean and has comments to make it easy for others to understand how it works. This program is useful for anyone who wants to sort and search stock information efficiently.